

PCBSegClassNet — A light-weight network for segmentation and classification of PCB component[☆]

Dhruv Makwana^a, Sai Chandra Teja R.^a, Sparsh Mittal^{b,*}

^a Independent Researcher, India

^b Electronics and Communications Engineering Department, Indian Institute of Technology (IIT) Roorkee, India

ARTICLE INFO

Dataset link: <https://github.com/CandleLabAI/PCBSegClassNet>

Keywords:

Computer vision
Deep learning
PCB component segmentation
Electronic component classification
Texture enhancement
Combined loss function

ABSTRACT

PCB component classification and segmentation can be helpful for PCB waste recycling. However, the variance in shapes and sizes of PCB components presents crucial challenges. We propose PCBSegClassNet, a novel deep neural network for PCB component classification and segmentation. The network uses a two-branch design that captures the global context in one branch and spatial features in the other. The fusion of two branches allows the effective segmentation of components of various sizes and shapes. We reinterpret the skip connections as a learning module to learn features efficiently. We propose a texture enhancement module that utilizes texture information and spatial features to obtain precise boundaries of components. We introduce a loss function that combines DICE, IoU, and SSIM loss functions to guide the training process for precise pixel-level, patch-level, and map-level segmentation. Our network outperforms all previous state-of-the-art networks on both segmentation and classification tasks. For example, it achieves a DICE score of 96.3% and IoU score of 92.7% on the FPIC dataset. From the FPIC dataset, we crop the images of 25 component classes and term the resultant 19158 images as the “FPIC-Component dataset” (we release scripts for obtaining this dataset from FPIC dataset). On this dataset, our network achieves a classification accuracy of 95.2%. Our model is much more light-weight than previous networks and achieves a segmentation throughput of 122 frame-per-second on a single GPU. We also showcase its ability to count the number of each component on a PCB. The code is available at <https://github.com/CandleLabAI/PCBSegClassNet>.

1. Introduction

As electronic items penetrate humans' lives, the amount of e-waste has increased rapidly. According to an estimate, nearly 50 million metric tonnes of electronic garbage is generated each year worldwide (Hadi, Xu, Lin, Hui, & McKay, 2015). Nearly 500 million computers were thrown away every year in the United States between the years 1997 to 2007 (Kiddee, Naidu, & Wong, 2013). Such massive amounts of electronic garbage need highly effective waste management strategies to minimize the negative environmental impact. Because electronic waste includes all types of electronic devices that are not classified, industrial procedures cannot recover them (Hadi et al., 2015). For these reasons, governments across the globe have developed standards for the proper disposal and management of electronic waste.

Printed circuit boards (PCBs) serve as a foundation for connecting both discrete and integrated electronic components. As such, discarded PCBs account for a large fraction of e-waste. PCBs contain many

precious metals and non-metals. According to an estimate, one ton of PCBs can provide nearly 284 g of gold (Li, Shrivastava, Gao, & Zhang, 2004). Barnwal and Dhawan (2020) note that by recycling 1 kg of ICs, 120 g ferrous metals (e.g., iron and nickel), 390 g metals (e.g., copper) and 490 g non-metals (e.g., silicon) can be recovered. Although the precious metals (such as gold, silver, and palladium) are, by weight, only 1% of the PCBs, they account for 80% of the total value of the PCBs (Awasthi, Zlamparet, Zeng, & Li, 2017). Thus, PCB recycling is beneficial to the environment and can generate high monetary value. However, due to the heterogeneous composition of PCB (Lee, Chang, Fan, & Chang, 2004; Li et al., 2004), it is challenging to forecast a PCB's material composition and develop recycling methods suited to that specific composition. Existing PCB recycling procedures can recover only a tiny fraction (e.g., 28 percent Li et al., 2004) of the dominant materials.

Manual recycling is time-consuming, expensive and thus, inefficient. A computer vision-based approach for PCB recycling is promising since

[☆] The computing systems used in this study were provided by the Indian Institute of Technology, Roorkee, India, under the grant FIG-100874. Dhruv worked on this paper while working as an intern at IIT Roorkee.

* Corresponding author.

E-mail addresses: dmakwana503@gmail.com (D. Makwana), saichandrateja@gmail.com (Sai Chandra Teja R.), sparsh.mittal@ece.iitr.ac.in (S. Mittal).

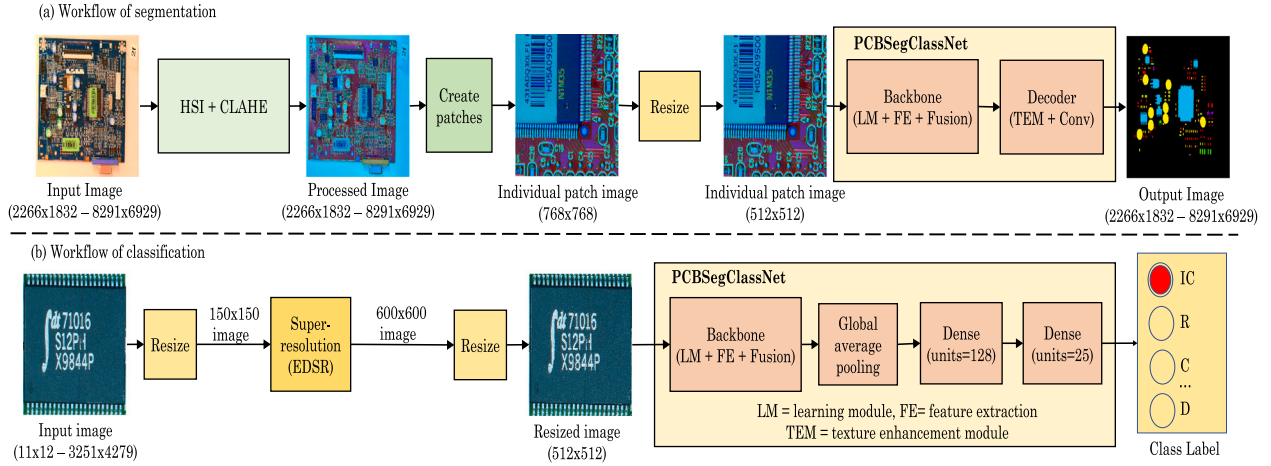


Fig. 1. The workflow of PCBSegClassNet for segmentation and classification tasks.

it is contact-free and avoids human hazards. The information obtained from the vision system can be fed to a desoldering system for automated disassembly, and recycling (Knoth, Hoffmann, Kopacek, & Kopacek, 2001). Given the huge success of deep learning for computer vision, a deep learning model can accurately identify PCB components. A few recent works (Kuo, Ashmore, Huggins, & Kira, 2019; Li, Gu, Huang, & Wen, 2019; Li, Li, Chen and Gu, 2021) have focused on detecting PCB components. However, these works do not perform component segmentation.

Contributions: In this paper, we present PCBSegClassNet, a novel deep neural network for PCB component segmentation and classification. We perform segmentation on the whole PCB image and classification on the individual component images (refer Sections 3.1–3.2). Fig. 1(a) shows the workflow of the segmentation task, and Fig. 1(b) shows the workflow of the classification task. For the segmentation task, we first convert the image to HSI color space and apply CLAHE processing (Section 3.3). Thus, we combine the best ideas from traditional computer vision and deep learning to achieve high predictive performance. To manage the compute overhead, we split the images into patches, resize them and feed them to our network's backbone. For classification, we first employ a super-resolution network to achieve high-resolution images. Then, we resize the images and feed them to the backbone of our network. The segmentation and classification networks share the same backbone. The difference is that the segmentation network has a decoder, whereas the classification network uses a global average pooling layer, followed by two dense layers. This shows the versatility of our network.

We design a novel two-branch network for achieving high segmentation and classification performance. Since PCB components vary in sizes and textures, the backbone in PCBSegClassNet uses a two-branch network design for effectively learning features from various size components. It captures the global context of components in one branch and spatial features in another. We effectively segment components of various sizes and shapes by fusing these two branches in the fusion module.

We use a novel texture enhancement module (TEM) in the decoder. PCB components have rich feature representations and strong edges. As such, both texture and spatial details are crucial for extracting semantics from the PCB board. Our proposed TEM utilizes texture information and spatial features to obtain precise boundaries of components. The TEM records long-term correlations using feature maps and then selectively aggregates a global context at areas with related textures. TEM significantly boosts intra-class uniformity and inter-class variance.

Most previous networks have been trained on a single loss function only. Since different loss functions have unique benefits, we propose to use a combined loss function named DIS ($\underline{DICE} + \underline{IoU} + \underline{SSIM}$) to bring

their best together. The DICE loss guarantees a good global minimum and promotes convergence in all pixels by giving equal weight to the foreground and background. The IoU loss focuses on big foreground regions, resulting in more homogeneous and confident probabilities for these areas. The SSIM loss takes into account each pixel's immediate neighborhood. It gives more weight to pixels in transitional buffer zones between foregrounds and backgrounds, such as borders and fine structures. In our proposed combined DIS loss function, the IoU loss function highlights the foreground and DICE loss keeps the gradient consistent across all pixels. By utilizing a greater loss around the borders, the SSIM loss encourages the prediction to maintain the structure of the original image while further pushing the background predictions to zero.

We summarize our contributions as:

- We propose a novel, light-weight network PCBSegClassNet for performing PCB component segmentation and classification in real-time. PCBSegClassNet uses a two-branch network design, a learning module and a fusion module to recognize PCB components of various sizes and shapes.
- We propose a Texture Enhancement Module (TEM) that leverages self-attention and global texture information to improve the representation semantics.
- We introduce a unique combined DIS loss that combines DICE, IoU, and SSIM loss functions to guide the training process for precise pixel-level, patch-level, and map-level segmentation.
- We are the first to present 25 class PCB component classification and segmentation results with bounding box and instances count on the FPIC dataset (Jessurun et al., 2022).
- We compare PCBSegClassNet with State-Of-The-Art (SOTA) networks on both segmentation (Badrinarayanan, Kendall, & Cipolla, 2017; Chen, Papandreou, Kokkinos, Murphy, & Yuille, 2017; Lin et al., 2017; Liu et al., 2021; Ronneberger, Fischer, & Brox, 2015; Wang et al., 2021, 2022; Zhao, Shi, Qi, Wang, & Jia, 2017) and classification (He, Zhang, Ren, & Sun, 2016; Huang, Liu, Van Der Maaten, & Weinberger, 2017; Liu et al., 2021; Mantravadi, Makwana, Teja, Mittal, & Singhal, 2023; Sandler, Howard, Zhu, Zhmoginov, & Chen, 2018; Tan & Le, 2019; Wang et al., 2021, 2022) tasks. We observe that our model provides superior results than previous techniques. PCBSegClassNet segmentation network has a model size of 5.58 MB and the classification network has a model size of 3.7 MB. Also, it achieves a throughput of 122 frames-per-second on RTX 2080Ti GPU for segmentation of 512 × 512 size images.
- From FPIC dataset, we crop the images of 25 component classes. This gives us 19158 component images with ground-truth. We term this dataset as the FPIC-Component dataset. We release scripts for obtaining FPIC-Component dataset from the FPIC

Table 1
Related works on PCB component classification, detection and segmentation.

Model	Network	Dataset/remark
Component classification		
Kim, Lee, and Park (2012)	HSI color model and K-means clustering	A PCB image
Cheong, Suandi, and Rahman (2019)	Faster-RCNN	Dummy PCB board images, 25 components
Polat, Tremeau, and Boochs (2021)	3D point cloud and hyperspectral imaging	Real electronic waste from landfill
Lim, Kim, and Park (2019)	CNN	12 481 images, 14 components
Mantravadi et al. (2023)	Dilated involutinal pyramid network	77,318 images from FICS-PCB dataset of 6 components
Component detection		
Li, Xu, Ran and Guo (2021)	SSD	500 images
Reza, Chen, and Crandall (2020)	YOLO, SSD	1500 images from internet
Li et al. (2019)	K-means++ clustering and modified YOLO-v3	WACV-2019 (Kuo et al., 2019)
Mallaiyan Sathiaselalan, Paradis, Taheri, and Asadizanjani (2021)	SSD, RCNN and a new network using region-proposal and similarity prediction	26 images
Júnior, Silva, Fernandes, Azevedo, and Oliveira (2022)	YOLO-v5, knowledge distillation	PCB-DSLR and WACV-2019 (Lu et al., 2020; Pramerdorfer & Kampel, 2015a)
Li, Li et al. (2021)	YOLO-v3 modified to match anchor size with receptive field size	Same as (Li et al., 2019)
Silva, Júnior, Azevedo, Oliveira, and Fernandes (2021)	YOLO-v3	PCB-DSLR (Pramerdorfer & Kampel, 2015a)
Component segmentation		
Pasunuri, Jessurun, Dizon-Paradis, and Asadizanjani (2021)	Comparison of UNet, DilatedNet, DeepLab, LinkNet, ICNet	8532 images, with masks of R,C,L,IC,Q
Li, Li, Chen, and Zhao (2020)	Random forest pixel classifier on depth difference features	200 synthetic images and some real images

dataset. We believe that it can catalyze future research in this field.

- We showcase the application of our technique for counting the exact count of each component on a PCB (Section 6.4). Once the individual components of a PCB are known, a database may be queried to assess its precious metal composition or produce a bill of material. Thus, our technique has immense potential for PCB and e-waste recycling.

The remainder of this article is organized as follows. Section 2 reviews recent works in PCB component identification. Section 3 discusses the dataset and pre-processing techniques. Section 4 explains our proposed network architecture and pre-training strategy. Section 5 presents the proposed loss function. Section 6 presents experimental results for segmentation and classification. Section 7 shows the ablation study and analyzes the failure cases to gain deeper insights. Finally, Section 8 concludes this work.

2. Related works

Related works on PCB component classification, detection and segmentation: Table 1 summarizes the key characteristics of recent works. Most techniques on component detection have used YOLO or SSD. Table 1 in the (Jessurun et al., 2022) summarizes the PCB-related datasets available in literature.

Related works on PCB/e-waste recycling: The PCB recycling techniques work at either (1) component-level or (2) entire PCB level. The component-level studies seek to identify components for which recycling is possible. Component identification studies are helpful for detailed analyses focusing on recycling individual parts, and the PCB may be indirectly identified via their individual parts. The works mentioned under the “whole PCB-level” seek to identify specific PCBs that can be recycled from the waste stream.

1. Component-level: Li, Esders, and Breier (2013) present an autonomous PCB recycling pipeline that uses several sensors, including lasers, cameras, and spectrometers for PCB analysis. They try to separate PCB components using the borders printed on the PCB substrate surrounding components. The limitation of their technique is that some PCBs may not have the border characteristics mentioned above. Li, Jiang, Breier, and Merhof (2016) provide an approach to locate components on a PCB by clustering areas with attributes including pixel

intensity, global texture information, and border edges. This information is further utilized for background removal. Weinert, Pandey, Naranjo Corona, and Danielewski (2017) present an automatic method for categorizing electronic garbage using deep learning. Putera and Ibrahim (2010) employ mathematical morphology and image processing techniques to determine the characteristics of certain kinds of manufacturing faults. They propose two methods for solving the PCB recycling dilemma. The first method seeks to determine individual recyclable parts at the component level. The second method explores whole PCB level component identification.

2. Whole PCB level: Pramerdorfer and Kampel (2015b) suggest a technique that uses local properties of PCB images along with homography verification to identify PCBs in a waste stream. Their technique obviates the requirement of extracting component-level data. The authors analyze and compare local feature matching approaches, such as “scale-invariant feature transform” (SIFT), “speeded up robust features” (SURF), and “oriented FAST and rotated BRIEF” (ORB). Their technique relies on a heuristically found threshold for every dataset. Due to this, porting their technique to another dataset may be challenging.

Related works on segmentation: PSPNet (Zhao et al., 2017) uses a pyramid parsing module that combines local and global information. It performs region-based context aggregation for leveraging global context information. UNet (Ronneberger et al., 2015) has a contracting path and an expansive path. The former follows the architecture of a CNN classifier. The latter semantically projects discriminative features onto the pixel space through upsampling and concatenation, followed by convolution.

FPN (Lin et al., 2017) generates multi-level feature maps proportionate to the input image, in a fully convolutional manner. It is especially useful for building feature pyramids in DNNs. SegNet’s (Badrinarayanan et al., 2017) novelty lies in upsampling encoder output. The decoder does non-linear upsampling by utilizing pooling indices computed in the max-pooling operation of the corresponding encoder. Deeplab’s (Chen et al., 2017) novelty lies in using a few atrous convolution layers towards the end of the backbone. Further, it has atrous Spatial Pyramid Pooling (ASPP) module, which uses dilated convolutions with different rates for capturing multi-scale context.

Vision-transformer has quadratic complexity with respect to the image size since it computes self-attention for the entire image. The Swin-transformer (Liu et al., 2021) generates hierarchical feature maps by using patch-merging in deeper layers. Then, it computes self-attention

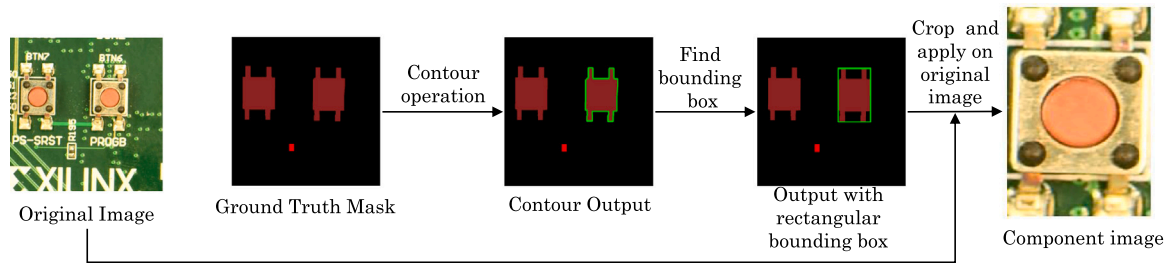


Fig. 2. Extracting component images from PCB images.

Table 2

25 PCB components that we target for segmentation. The “count” column shows the number of images of that component in the FPIC-component dataset.

No.	Component	Count	No.	Component	Count	No.	Component	Count
1	Resistor (R)	7246	9	Resistor Network (RN)	330	18	Switch (SW)	50
2	Capacitor (C)	6896	10	Test Point (TP)	266	19	Transformer (T)	47
3	Integrated Circuit (U)	761	11	Integrated Circuit (IC)	237	20	Fuse (F)	44
4	Discrete Transistor (Q)	616	12	Plug (P)	200	21	Vacuum Tube (V)	41
5	Connector (J)	579	13	Thyristor (CR)	194	22	Light Emitting Diode (LED)	39
6	Inductor (L)	473	14	Motor (M)	74	23	Switch (S)	37
7	Resistor Coil (RA)	404	15	Button (BTN)	72	24	QA	36
8	Diode (D)	362	16	Ferrite Bead (FB)	69	25	Jumper Link (JP)	31
			17	CRA	54			

only inside the local window; hence, it only has linear complexity. PVT (Wang et al., 2021, 2022) uses a progressive shrinking pyramid, such that the output resolution of its four stages reduces from high (four-stride) to low (32-stride). This reduces the number of computations. It also uses a “spatial-reduction attention” layer for efficiently learning high-resolution features.

3. Dataset and pre-processing techniques

3.1. FICS PCB Image Collection (FPIC) dataset for segmentation

We evaluate the performance of our method on publicly available dataset FPIC (Jessurun et al., 2022). There are 230 images of 73 distinct PCBs in the dataset, taken from both the front and rear sides of the PCBs. Over 58,000 annotations are included in the images, including annotations for the text and any attached components. There are total of 58 different components annotated. Of these, we use 25 components for evaluation. We excluded the remaining components since only up to 5 images of those components were present. Table 2 shows the name of these 25 components.

3.2. Obtaining component images from PCB images

The FPIC dataset provides images of the whole PCB. For performing component-level classification, we create a new dataset named FPIC-component, which contains component images cropped from the PCB images. In Table 2, the column “count” shows the number of images of that component in FPIC-component dataset.

Fig. 2 shows the steps taken to crop the component images; they are as follows:

- Perform contour operation on the ground truth mask to find the boundary of each component.
- From contour and boundary information, find a rectangular bounding box that covers the whole contour for every component.
- From the rectangular bounding box, crop the component region from the original image.
- Color information from the ground truth mask in the contour region provides us with the ground truth label for the corresponding PCB component.

Table 3

The distribution of height and width (in pixels) for the images in FPIC-component dataset.

Pixel range	Height	Width
0–50	10 401	7412
50–100	5408	8221
100–150	1122	1355
150–500	1724	1650
500–1000	398	410
1000–10 000	105	110

Table 3 shows the height/width distribution of images in FPIC-component dataset. Notice that the size of PCB components varies widely and many components span fewer than 50 pixels. This, along with high integration density, highlights the challenges in and importance of segmentation.

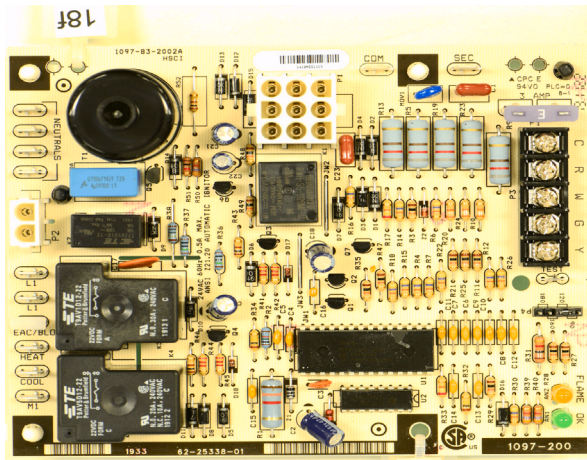
3.3. Conversion to HSI space and CLAHE preprocessing

The conventional deep neural networks operate on images stored in RGB color space. However, Red (R), Green (G), and Blue (B) are not independent constituents of the color space, and hence, evaluating the image using the RGB color space captures limited feature representations.

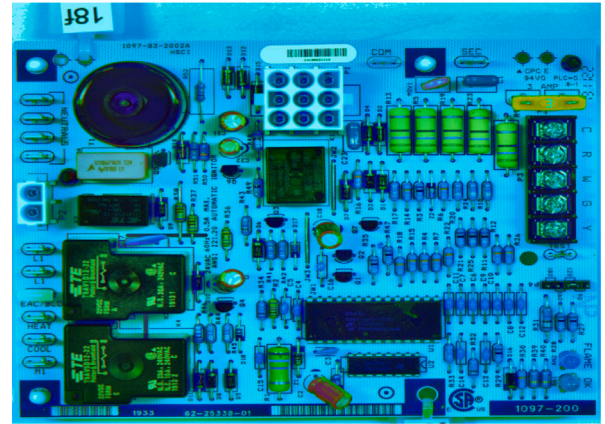
On the other hand, in the HSI color space, the three components of an image’s color representation, namely, hue (H), saturation (S), and intensity (I), are decoupled. Another benefit of the HSI color space is that the hue remains consistent regardless of lighting conditions. We use HSI color space with the CLAHE method (Zuiderveld, 1994) for pre-processing the PCB image. CLAHE is an advanced method for adaptive histogram equalization. It prevents the over-amplification of noise resulting from adaptive histogram equalization. Fig. 3 shows a sample PCB image and its corresponding image in HSI color space after applying the CLAHE method.

3.4. Use of patches for segmentation

The resolution of PCB images in the FPIC dataset varies from 2266×1832 to 8291×6929 . Thus, the highest resolution image has a resolution higher than that of even an 8K UHD (ultra-high definition) image. Since processing large images can overwhelm the



(a) RGB image



(b) HSI + CLAHE image

Fig. 3. Preprocessing of PCB image with HSI color space and CLAHE.

GPU, we operate on image patches instead of the entire image. We have experimented with the patches of sizes 256×256 , 512×512 , 768×768 , and 1024×1024 . The results obtained with patches of size 1024×1024 and 768×768 were found to be comparable, whereas smaller patch sizes provided inferior results. Hence, we choose a patch size of 768×768 , which captures the most representative sections without overwhelming the GPU. From this, we obtain 5048 patch images for training and 1262 patch images for testing. We use the bilinear interpolation method to resize every patch to 512×512 size image and use them for training and inference. During inference, the masks are predicted for individual patches. We combine the masks from multiple patches to create the mask for the whole image.

3.5. Performing super-resolution

The PCB component images have diverse shapes and resolutions. We apply super-resolution to obtain high-resolution images to learn fine-grained details and perceive features effectively. High-resolution images are more detailed, making it easier to see even the smallest features. For super-resolution, we first considered basic upsampling. However, even using bilinear interpolation, basic upsampling cannot extract additional information from network training. Hence, we use “Enhanced Deep Residual Networks for Single Image Super-Resolution” (EDSR) (Lim, Son, Kim, Nah, & Mu Lee, 2017) that has been pretrained on DIV2K (Agustsson & Timofte, 2017) dataset. EDSR architecture is inspired by the SRResNet (Ledig et al., 2017) architecture and is made up of multiple residual blocks. Consistent results are achieved by using constant scaling layers instead of batch normalization layers. Fig. 4 illustrates the effect of super-resolution. For the given IC (integrated circuit) image of size 150×150 , the EDSR network generates a higher resolution image of size 600×600 with more detailed information.

4. Architecture of our proposed PCBsegClassNet

4.1. Motivation and key ideas

PCB component segmentation presents unique challenges due to the high variance in PCB component sizes and shapes (Mallaiyan Sathiaselan et al., 2021). The segmentation task differs from the classification task because it requires more local information based on high-resolution images and global characteristics in low-resolution to discriminate between foreground and background (Zeng, Xie, Zhang, & Lu, 2019).

Researchers have used encoder-decoder networks for segmentation, which utilize information from various levels and resolutions. In the encoder, the early layers extract information about low-level details such as component edges. The deeper layers extract higher-level semantic information and distinguish between different components. Recent works have used encoder-decoder networks with skip connections, and two branches (Luo & Yang, 2020; Mazzini, 2018; Nag, Makwana, R, Mittal, & Mohan, 2022; Ouali, Hudelot, & Tami, 2020). After learning global information from the low-resolution version of the input image, convolution neural networks with full input resolution are used to improve segmentation results. Efficient feature learning from small components (such as resistors, and capacitors) is not possible using skip-connection in early layers since they extract low-level characteristics.

To tackle the challenges mentioned above, we propose PCBsegClassNet, a novel segmentation and classification network. To effectively learn features of both small size and large size, we advocate the use of a “learning module” and two branch network. We reinterpret the skip connections as a learning module to learn features efficiently. Our two-branch network focuses on capturing global context in one branch and learning spatial features in the other. The fusion of two branches allows effective segmentation of components of various sizes and shapes in real-time.

The inference latency of a network depends primarily on input resolution and network depth. To keep the input resolution low, we create patches of size 768×768 from the image (Section 3.4), and then, resize them to 512×512 . The encoder in our network operates on 512×512 images because, with this size, even small component information can be retained. In each layer of the network, the number of filters are multiple of 16. These design choices lead to a light-weight network and help us achieve real-time performance.

Fig. 5 shows the architecture of PCBsegClassNet, which follows the encoder-decoder architecture. We now explain the architecture in more detail.

4.2. Encoder

The encoder uses a learning module, a feature extractor and a fusion module. We now describe these modules.

Learning module: Our proposed learning module shares feature computation between the low and high-level branches in a network. It extracts local details by employing a series of three layers. To ensure the effective sharing of low-level features, just three layers are used. The first layer is a regular convolutional layer (called Conv), and

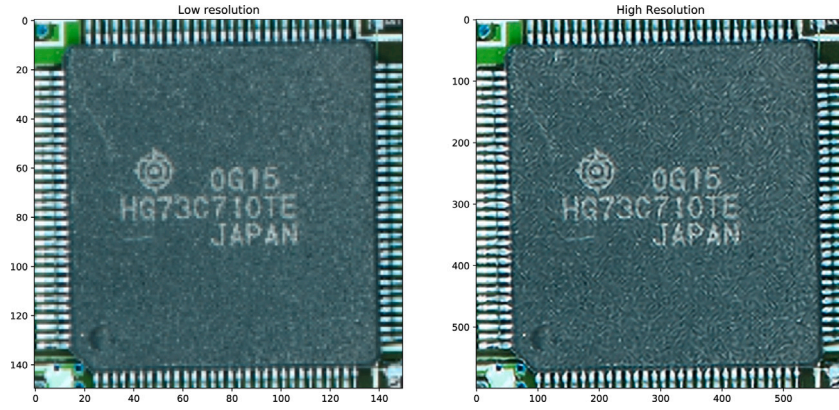


Fig. 4. A low-resolution (150×150) input image and the corresponding high-resolution (600×600) image generated by EDSR.

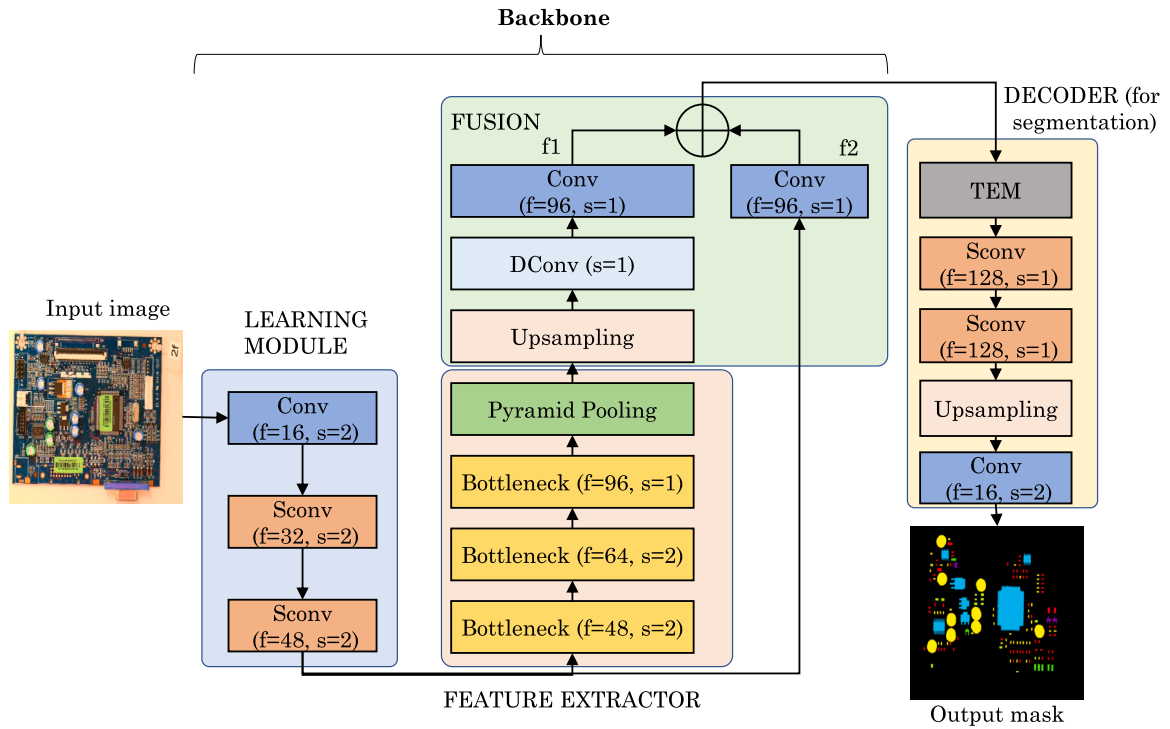


Fig. 5. Architectural diagram of PCBSegClassNet for segmentation (s = stride, f = number of filters). For classification, the decoder is replaced by a GAP layer and 2 dense layers, as shown in Fig. 1.

the subsequent two layers are separable convolutional layers (SConv). Many recent networks have also used SConv layers (Chollet, 2017; Howard et al., 2017). A stride value of two and a kernel size of 3×3 is used for all three layers in the learning module, followed by batch normalization and ReLU nonlinearity.

Feature extractor module: The feature extractor (FE) module strives to capture the global context. The FE module consists of three bottleneck residual blocks and a pyramid pooling block. For the bottleneck residual blocks, the residual connection has the same input and output size. The bottleneck residual block converts the feature map from c channels to $\hat{c}$ channels. We use the residual bottleneck module proposed by MobileNetV2 (Sandler et al., 2018). The bottleneck block consists of 1×1 Conv, followed by 3×3 SConv, followed by 1×1 Conv. All three convolution layers have ReLU nonlinearity. We use SConv to reduce the number of parameters and floating point operations. The pyramid pooling module (Zhao et al., 2017) is introduced at the end to aggregate the context information from various regions. The pyramid pooling module combines four distinct pyramid scales.

Fusion module: In the fusion module, we first apply bilinear up-sampling on feature extractor output followed by depthwise convolution (DConv) and standard convolution to generate the feature $f1$ (refer to Fig. 5). We apply 1×1 convolution on the output of the learning module and obtain the feature $f2$. The features $f1$ and $f2$ have the same number of channels. In the fusion module, we add features $f1$ and $f2$ and then normalize the fused features by batch normalization layer with ReLU activation.

4.3. Proposed texture enhancement module

Motivation: For PCB component recognition, texture plays an important role. The texture is critical in the extraction of semantics from the PCB board. Spatial details and semantic features are also important for the segmentation of PCB components. PCB components have rich feature representations and strong edges. Various channels of the feature map represent various sorts of texture information of components. Efficient low-level texture information learning improves

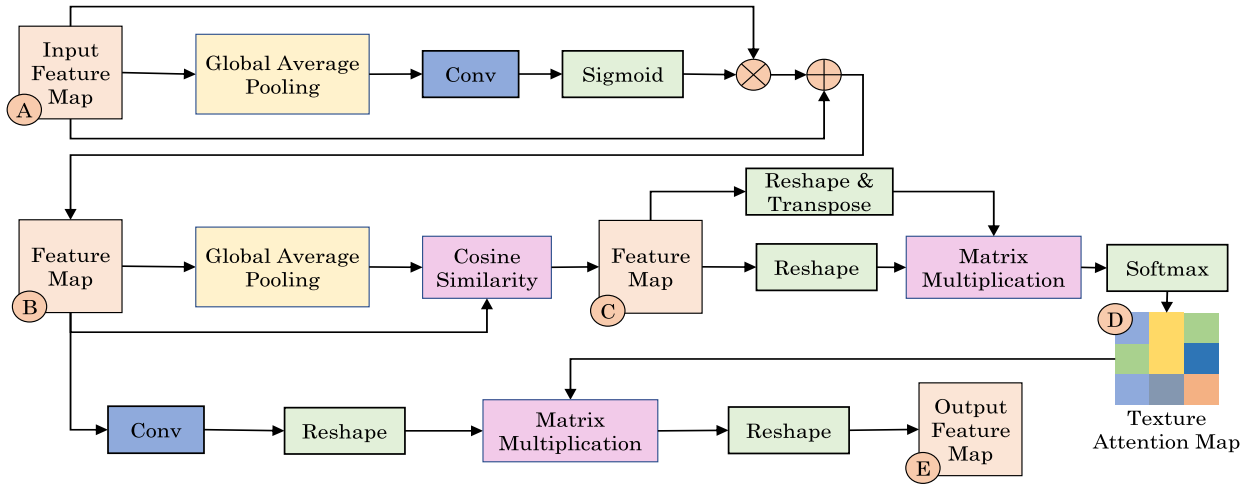


Fig. 6. Texture enhancement module (TEM).

the segmentation of components. For this purpose, we propose a texture enhancement module (TEM) that utilizes the texture information and spatial features to obtain precise boundary of components. The TEM records long-term correlations using feature maps and then selectively aggregates a global context at areas with related textures. When it comes to semantic extraction, TEM not only provides a bird's-eye view of the feature map but also vastly enhances intra-class uniformity and inter-class variance. Further, it increases the performance of semantic branches.

Design of TEM: Our proposed TEM is illustrated in Fig. 6. In order to produce the feature map (B) $\in \mathbb{R}^{C \times H \times W}$ in Fig. 6, we start with a channel attention operation together with residual connection to enhance feature map (A) channel representation. We first perform Global Average Pooling (GAP) on the feature map (A), which then passes through convolution layer. The convolution is followed by sigmoid activation. The resultant feature map is multiplied and added to the feature map (A).

Once the weights are recalculated, the channels representing crucial textures in feature map (B) stand out more prominently than the ones in feature map (A). GAP is applied on feature map (B) to obtain the feature map $g \in \mathbb{R}^{C \times 1 \times 1}$. We compute the cosine similarity (Zhu et al., 2021) between feature map g and every spatial dimension of feature map (B), which results in feature map (C) $\in \mathbb{R}^{H \times W}$. The cosine similarity computation is shown in Eq. (1). Here, the spatial dimension of the feature map is (i, j) . Based on this feature map, we construct a self-attention module.

$$D_{i,j} = \frac{g \cdot B_{i,j}}{\|g\|_2 \cdot \|B_{i,j}\|_2} \quad (1)$$

The feature map (C) is reshaped into $\mathbb{R}^{C \times N}$, where $N = H \times W$. A matrix multiplication operation is performed between the transpose of feature map (C) and feature map (C) itself. Then, a softmax activation is used to get a texture attention map (D) $\in \mathbb{R}^{N \times N}$. (D) shows correlations of texture at different spatial locations in feature map (B). In Eq. (2), S represents self-attention, whereas $\cdot$ is matrix multiplication operation.

$$S = \text{softmax}(\text{trans}(T) \cdot T) \quad (2)$$

Feature map (B) is passed through convolution and reshaped into $\mathbb{R}^{C \times N}$. Then, the resulting feature map matrix is multiplied with feature map (D) and reshaped to obtain the final feature map (E) $\in \mathbb{R}^{C \times H \times W}$.

4.4. Decoder

The decoder has texture enhancement module (TEM) followed by two separable convolution layers. We observe that the accuracy can

be improved by adding a few more convolution layers following the feature fusion module. The outcome of separable convolution is passed through an upsample layer to get the exact size output as the input image. At last, we have a 1×1 convolution layer to generate the mask for 25 classes.

The network for classification task: In Fig. 5, we get the classification network by replacing the decoder module with global average pooling followed by 128 and 25 units dense layers.

4.5. Pre-training

The high variance in the shapes and textures of the PCB component makes the segmentation challenging. We overcome this challenge by pre-training our encoder on the proposed FPIC-component dataset, which boosts segmentation results. Pre-training is carried out on high-resolution images obtained using super-resolution with the EDSR network (Section 3.5). We pre-train our network encoder for 100 epochs with N-pair contrastive loss (Sohn, 2016) to learn better feature representation of component data. We then use the learned feature representations from the pre-trained weights to train segmentation model.

The “cross-entropy loss” is the most often employed loss function in supervised learning. However, it has numerous drawbacks (Zhang & Sabuncu, 2018), such as noisy labels, which cause a lack of resilience and poor performance with adversarial cases. In this paper, we utilize N-pair contrastive loss for supervised learning. It ensures that normalized embeddings from the same category are pulled together more closely than embeddings from other categories. This loss is used to train the encoder to build vector embeddings of component images so that representations of images in the same component class are more similar than representations of images in other component classes. Our experiments confirm that N-pair contrastive loss outperforms supervised training with “cross-entropy loss”. Pretraining an encoder with N-pair contrastive loss results in improved embedding representation in the latent dimension of PCB components with varying shapes and textures. It minimizes the distance between comparable embeddings, improving feature learning in the encoder stage.

5. Proposed combined DIS loss function

We first define various loss functions (Section 5.1) and then present our proposed combined loss function (Section 5.2).

5.1. Background on loss functions

1. DICE loss: DICE loss is the most commonly used loss in segmentation. If y is the ground-truth mask and $\hat{y}$ is the predicted mask, then DICE coefficient loss (Fidon et al., 2017) is defined as:

$$\mathcal{L}_{DICE} = \frac{2 * |y \cap \hat{y}|}{|y| + |\hat{y}|} \quad (3)$$

2. IoU loss: Intersection over Union (IoU) is a standard performance metric for object detection and segmentation. The IoU loss (Máttyus, Luo, & Urtasun, 2017) is defined as:

$$\mathcal{L}_{IoU} = 1 - \frac{|y \cap \hat{y}|}{|y \cup \hat{y}|} \quad (4)$$

3. SSIM loss: Structural SIMilarity (SSIM) is used to measure an image's structural information. Let the ground truth mask pixel values be $y = \{y_j : j = 1, \dots, M^2\}$, and the predicted mask pixel values be $\hat{y} = \{\hat{y}_j : j = 1, \dots, M^2\}$. The SSIM loss (Wang, Simoncelli, & Bovik, 2003) of y and $\hat{y}$ is defined as:

$$\mathcal{L}_{SSIM} = 1 - \frac{(2\mu_y\mu_{\hat{y}} + C_1)(2\sigma_{y\hat{y}} + C_2)}{(\mu_y^2 + \mu_{\hat{y}}^2 + C_1)(\sigma_y^2 + \sigma_{\hat{y}}^2 + C_2)} \quad (5)$$

where $\mu_y, \mu_{\hat{y}}$ and $\sigma_y, \sigma_{\hat{y}}$ are mean and standard deviation of y and $\hat{y}$ respectively, $\sigma_{y\hat{y}}$ is covariance, C_1 , and C_2 are constants which are used to avoid dividing by 0.

5.2. Our proposed loss function

Most previous networks have been trained on a single loss function only. Since different loss functions have unique benefits, we propose to use a combined loss function, named DIS ($\mathcal{L}_{DICE} + \mathcal{L}_{IoU} + \mathcal{L}_{SSIM}$). It is defined as

$$\mathcal{L}_{DIS} = \mathcal{L}_{DICE} + \mathcal{L}_{IoU} + \mathcal{L}_{SSIM} \quad (6)$$

The DICE loss guarantees a good global minimum and promotes convergence in all pixels by giving equal weight to the foreground and background. IoU is a map-level measurement. As a result, the models trained with IoU loss focus more on foreground regions, resulting in more homogeneous and confident probabilities for these areas. The SSIM loss takes into account each pixel's immediate neighborhood. It gives more weight to pixels in transitional buffer zones between foregrounds and backgrounds, such as borders and fine structures. SSIM loss helps in identifying PCB components based on their structural information.

In our proposed combined DIS loss function, the IoU loss function highlights the foreground and DICE loss keeps the gradient consistent across all pixels. By utilizing a more significant loss around the borders, the SSIM loss encourages the prediction to maintain the structure of the original image while further pushing the background predictions to zero. Overall, the DIS loss function helps PCBSegClassNet achieve high-quality segmentation maps and clear boundaries.

6. Experimental results

We first discuss the experimental platform (Section 6.1) and then present results on segmentation task (Section 6.2) and classification task (Section 6.3). We finally showcase the use of our technique for PCB component counting (Section 6.4).

6.1. Implementation setting and metrics

We use TensorFlow and CUDA 11.2. The initial learning rate is 0.001. When there is no convergence after 15 consecutive epochs, the decay rate of 0.1 is applied to the initial learning rate. The batch size is 16. The encoder is pre-trained for 100 epochs using Adam optimizer and N-pair contrastive loss. We then use the Adam optimizer with our proposed DIS loss for training the segmentation network and

categorical cross-entropy loss for training the classification network for 100 epochs. The parameters are initialized by the glorot-uniform method (Glorot & Bengio, 2010).

For all the networks, we use the same training and testing data, super-resolution technique, hyperparameters, same “combined DIS loss function” and HSI + CLAHE processed images.

We employ accuracy, Matthews Correlation Coefficient (MCC) (Chicco & Jurman, 2020), precision, recall, and F1-score measures to evaluate classification performance. MCC is defined in Eq. (7), where TP, TN, FP, and FN states True Positive, True Negative, False Positive, and False Negative, respectively.

$$MCC = \frac{TP * TN - FP * FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (7)$$

Notice that MCC considers all four (TP/TN/FP/FN) into account. MCC is the only binary classification rate that has a high score only if the binary classifier can correctly predict most of the positive instances and most of the negative instances. To assess the segmentation performance, we employ “IoU” and “DICE coefficient”.

6.2. Segmentation results

Quantitative Results: Since PCBSegClassNet is the first model to show segmentation results on the FPIC, we compare it to eight state-of-the-art open-source models, namely, PSPNet (Zhao et al., 2017), UNet (Ronneberger et al., 2015), FPN (Lin et al., 2017), SegNet (Badrinarayanan et al., 2017), DeepLabv3+ (Chen et al., 2017), PVT-small (Wang et al., 2021), PVT2-B0 (Wang et al., 2022), and Swin-Transformer (Liu et al., 2021).

The quantitative results are depicted in Table 4. Under the category “without pretraining”, the results have been obtained without pretraining the networks on FPIC-component dataset. Thus, these networks have been trained on FPIC dataset from scratch. For obtaining the results under the category “using pretraining”, we have first pretrained the networks on FPIC-component dataset and then trained them on the FPIC dataset. Only the top four networks (on DICE score) in “without pretraining” category have been selected for showing results under “using pretraining” category.

Results without pretraining: PCBSegClassNet provides the highest IoU and has the smallest model size. In fact, the model size of PCB-SegClassNet is nearly 17.9× smaller than that of PVT2-B0. On DICE score, it is only a little behind PVT2-B0. It is noteworthy that due to the large model size, PVT-small and PVT2-B0 require much higher training time and iterations to converge.

Results with pretraining: Pretraining improves the DICE score of all the networks, which underscores its importance. PCBSegClassNet has comparable DICE score and better IoU than PVT2-B0. PCBSegClassNet achieves high IoU by virtue of using the proposed texture enhancement module. Considering the balance between model size, and predictive performance metrics, PCBSegClassNet can be taken as the best model. Note that all the segmentation results of PCBSegClassNet shown later in this paper have been obtained with pretraining.

Throughput and model size: For a 512×512 size image, our segmentation network has a throughput of 122 frames-per-second. By contrast, PVT2-B0 has an FPS of 98. Our model is light-weight with 1.31M parameters and a 5.58 MB size. As seen from Table 4, our network is much lighter than the previous networks. On replacing separable convolution and depthwise convolution with standard convolution, the number of parameters in PCBSegClassNet increases to 1.9M and the model size increases to 7.89 MB. This validates the use of separable and depthwise convolution in our model.

Qualitative results: Fig. 7 shows the qualitative comparisons against several state-of-the-art networks. As can be seen, PCBSegClassNet is flexible enough to deal with a wide variety of challenging images, including those involving tiny objects (i), huge objects with complex borders (ii), few objects (iii), and many objects (iv). UNet

Table 4
Segmentation results and model size of various networks.

Model	DICE	IoU	Parameters (M)	Model size (MB)
Without pretraining on FPIC-component dataset				
PSPNet (Zhao et al., 2017)	88.09	79.66	10.12	40.59
UNet (Ronneberger et al., 2015)	88.37	66.23	31.40	125.78
FPN (Lin et al., 2017)	89.64	77.71	17.60	70.55
SegNet (Badrinarayanan et al., 2017)	90.35	76.68	29.46	118.06
DeepLabv3+ (Chen et al., 2017)	90.97	76.91	41.25	166.14
Swin-Transformer (Liu et al., 2021)	90.55	83.60	86.13	357.78
PVT-small (Wang et al., 2021)	91.61	83.23	44.10	176.62
PVTv2-B0 (Wang et al., 2022)	93.96	88.24	23.50	96.82
PCBSegClassNet	93.47	90.33	1.31	5.58
Using pretraining on FPIC-component dataset				
DeepLabv3+ (Chen et al., 2017)	92.27	85.14	41.25	166.14
PVT-small (Wang et al., 2021)	94.67	89.13	44.10	176.62
PVTv2-B0 (Wang et al., 2022)	96.31	92.51	23.50	96.82
PCBSegClassNet	96.29	92.74	1.31	5.58

Table 5

Comparison of classification accuracy, MCC, precision, recall, F1-Score, number of parameters, and model size of different models.

Model	Accuracy	MCC	Precision	Recall	F1-Score	Parameters (M)	Model-size (MB)
DenseNet121 (Huang et al., 2017)	71.29%	58.51%	66.82%	71.29%	67.87%	7.06	29.23
MobileNetV2 (Sandler et al., 2018)	88.93%	84.75%	91.91%	88.93%	89.73%	2.29	9.56
ResNet50 (He et al., 2016)	88.50%	83.91%	91.70%	88.50%	88.39%	23.63	94.99
EfficientNetB0 (Tan & Le, 2019)	91.94%	88.61%	91.42%	91.94%	91.48%	4.08	16.86
DInPNet (Mantravadi et al., 2023)	92.88%	90.13%	93.53%	92.88%	93.00%	0.53	2.03
Swin-Transformer (Liu et al., 2021)	90.89%	87.26%	91.17%	90.89%	90.41%	50.0	101.3
PVT-small (Wang et al., 2021)	91.89%	88.59%	91.60%	91.89%	91.51%	24.5	96.77
PVTv2-B0 (Wang et al., 2022)	93.85%	91.41%	94.59%	93.85%	93.75%	3.66	14.04
PCBSegClassNet	95.16%	93.20%	95.53%	95.16%	95.22%	0.84	3.70

and DeepLabv3+ fail to cover the structure and shape of the object. In column (ii), UNet and DeepLabv3+ are unable to segment various pins of ICs properly. By contrast, PCBSegClassNet shows the shape of ICs with all pins properly with a clear, small distance between all the pins.

From column (i), we note that PSPNet finds challenges in segmenting small objects and FPN misclassifies rarely occurring objects with more than one mask color (shown with red circle). Further, from column (i), we observe that the masks generated by FPN and DeepLabv3+ fail to segment the ground truth “IC” class on the PCB Image. From the “IC” class, half of the pixels are classified as “IC” class and another half as “SW” class. Compared to FPN and DeepLabv3+, PCBSegClassNet correctly classifies objects of all sizes.

Column (iii) shows the extra component ‘J’ (shown in orange) predicted by UNet and DeepLabv3+ below the center IC, which does not exist in the ground truth. For the image in column (iv), FPN, PSPNet, and SegNet mispredict some pixels in vertical line. This is due to combining the mask of individual patches to create the mask for the entire PCB. Despite combining various patches, PCBSegClassNet does not miss any pixel in any patch boundaries, which shows the robustness of our proposed architecture.

6.3. Classification results

Average results: We compare the performance of PCBSegClassNet with eight SOTA classification networks, viz., ResNet50 (He et al., 2016), MobileNetV2 (Sandler et al., 2018), EfficientNetB0 (Tan & Le, 2019), DenseNet121 (Huang et al., 2017), DInPNet (Mantravadi et al., 2023), PVT-small (Wang et al., 2021), PVTv2-B0 (Wang et al., 2022), and Swin-Transformer (Liu et al., 2021). Note that all the networks, including ours, have been trained from scratch on FPIC-component dataset. Table 5 shows the results. PCBSegClassNet outperforms other models on all the predictive performance metrics. Its high F1 score suggests that PCBSegClassNet can effectively capture the fine structures from the PCB images.

An L-layer DenseNet has a total of $(L(L + 1))/2$ connections. Excessive connections reduce network processing and parameter efficiency

and render networks more susceptible to overfitting. Hence, increasing the network depth of DenseNet increases memory usage while decreasing accuracy. Due to this, DenseNet achieves a low accuracy (71.29%). Among the light-weight models, MobileNetV2 has lower accuracy of 88.93%, while EfficientNetB0 shows an accuracy of 91.94%. This shows the superiority of the network architecture of EfficientNetB0 over that of MobileNetV2.

DInPNet, which uses dilated involutonal pyramid block, achieves only 92.88% accuracy. Yet, it is the most lightweight network. Among the transformer networks, PVTv2-B0 has the highest accuracy, viz., 93.85%. PCBSegClassNet surpasses all the networks and achieves an accuracy of 95.16%. It is also the best on MCC, precision, recall and F1-score.

As discussed in Section 4, we have specifically optimized our network architecture and hyperparameters for PCB component classification and segmentation. The SOTA models use max-pooling for downsampling. By contrast, we use convolution with a stride of two in the learning module. This enables a more effective extraction of features of components of all sizes. PCBSegClassNet extracts features at four different pyramid scales by combining the strengths of the bottleneck block used by MobileNetV2 and the pyramid pooling used by PSPNet. While SOTA networks have been developed for a broad range of computer vision tasks, PCBSegClassNet uses TEM for texture enhancement, which is crucial for PCB component segmentation.

PCBSegClassNet has only 0.84M parameters and model size of 3.70 MB, which signifies its usefulness in edge devices. In fact, the model size of PCBSegClassNet is $\sim 26\times$ smaller than that of ResNet50. On accounting for the balance between accuracy and model size, PCBSegClassNet is much superior to the SOTA networks.

Classwise results: Table 6 shows the classwise results of the five networks that show highest accuracy. In the FPIC-Component dataset, the number of samples varies significantly across different categories, as shown in Section 3. Hence, we show both macro average and weighted average. The macro average score does not consider the proportion for each class in the dataset, while the weighted average considers the proportion for each class in the dataset. The weighted average

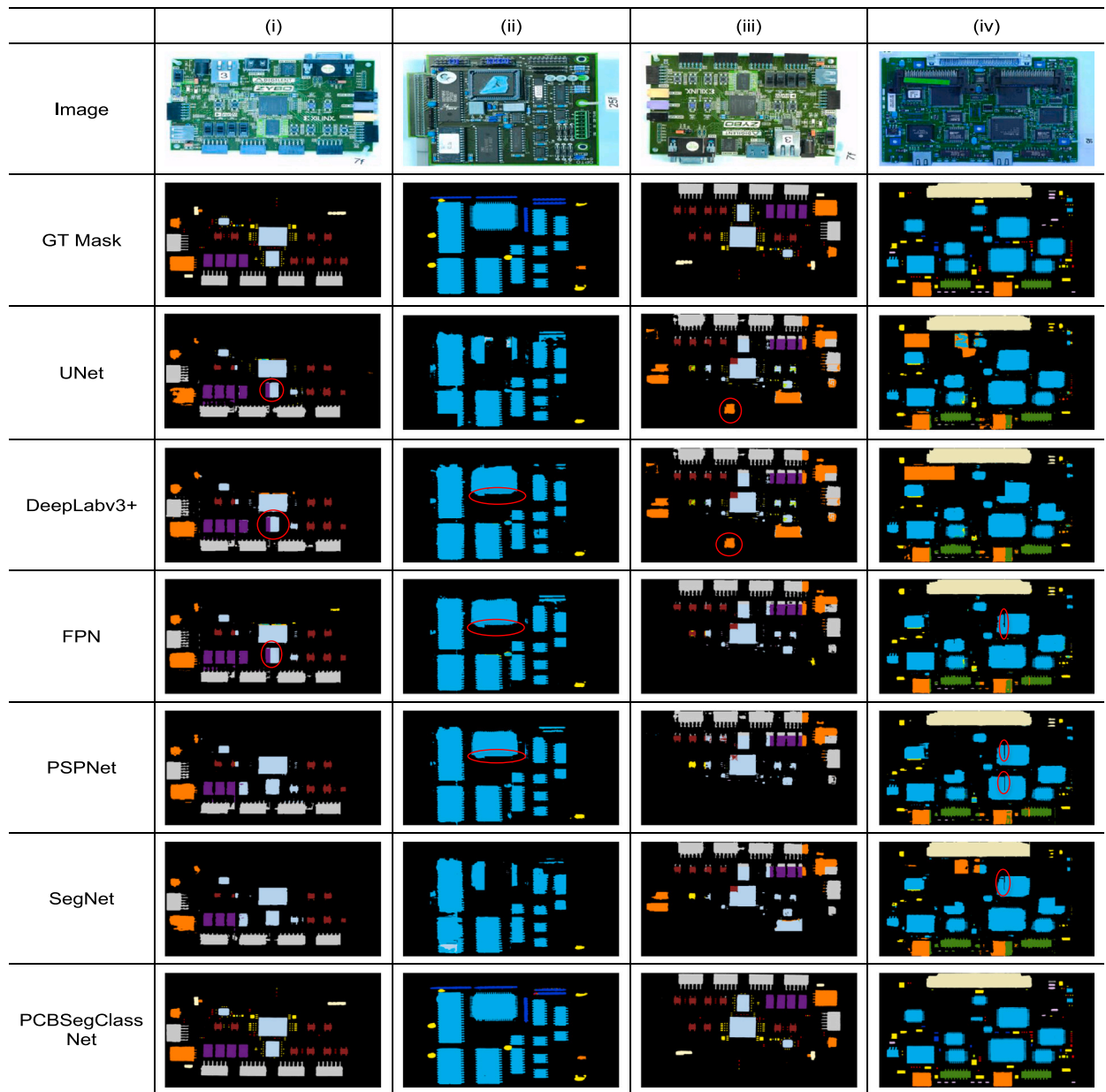


Fig. 7. Qualitative results of PCBSegClassNet on FPIC. first row: Input Image, second row: ground truth, third row: UNet predicted mask, fourth row: DeepLabv3+ predicted mask, fifth row: FPN predicted mask, sixth row: PSPNet predicted mask, seventh row: SegNet predicted mask, last row: PCBSegClassNet predicted mask.

score presents a fair comparison in an imbalanced dataset, while macro average score shows the ability of the model to overfit towards the class with more sample data. Our network is the best on both macro and weighted average scores on the F1-score metrics. We have also examined the precision and recall metrics and found that our network is the best on those metrics also.

We now comment on the results of selected classes. For the “M” class, PVTv2-B0 has the best F1-score of 86.7%, compared to PCB-SegClassNet, which scores 62.2%. This is due to intra-class variability and inter-class similarity with “IC” and “U” classes. Deeper networks like EfficientNetB0 attain high score for classes with similar-looking features. For example, for “FB” class, F1-score with both EfficientNetB0 and PVTv2-B0 is 72.7%, whereas that with PCBSegClassNet is 50%. Out of the 69 images of the “FB” class, most images look similar to images of the “C” and “L” classes. Similarly, PVTv2-B0 and PVT-Small show better feature learning for classes with few samples; this can be seen for the LED class, which has only 39 images.

For classes with intra-class variability, PCBSegClassNet shows superior value of F1-score. For example, there are only 31 images of the “JP” class, and these images show intra-class variability. For “JP” class, the F1-score is 44.4% with PCBSegClassNet, zero with EfficientNetB0, DInPNet and PVT-Small, and only 25% with PVTv2-B0.

6.4. Application of our technique for PCB component counting

Detection of PCB components is helpful for various purposes, such as producing a bill of materials and finding the components that can be recycled. We now show object detection for performing component counting. We follow the following steps to draw a bounding box on the PCB image. First, we create patches of query PCB image and perform inference on patches to get segmentation mask. We combine the patches’ masks to create the combined PCB image mask. Once we get the whole PCB image mask, we perform a contour operation to get the boundary of each Component. Using the boundary information of each Component, we draw a bounding box around the Components. We use

Table 6

Classwise F1-score comparison (avg = average).

Class	EfficientNetB0	DInPNet	PVT-Small	PVTv2-B0	PCBSegClassNet
R	98.20%	98.22%	97.14%	98.17%	98.34%
C	97.14%	95.79%	95.78%	97.52%	98.13%
U	79.62%	83.17%	84.39%	84.24%	86.78%
Q	71.30%	85.60%	78.97%	84.85%	86.96%
J	83.51%	92.00%	79.02%	74.81%	86.70%
L	73.27%	74.80%	80.00%	88.76%	89.45%
RA	93.83%	100.00%	100.00%	100.00%	99.35%
D	56.72%	60.32%	47.62%	40.00%	72.13%
RN	81.32%	93.07%	96.91%	96.91%	94.95%
TP	95.74%	88.07%	91.09%	96.97%	97.96%
IC	57.14%	71.15%	62.65%	77.78%	84.78%
P	58.82%	66.67%	66.67%	59.57%	66.67%
CR	61.36%	81.48%	69.84%	67.26%	86.84%
M	64.52%	71.79%	73.68%	86.67%	62.22%
BTN	84.85%	100.00%	84.21%	100.00%	96.97%
FB	72.73%	45.45%	40.00%	72.73%	50.00%
CRA	50.00%	0.00%	0.00%	95.65%	86.96%
SW	53.33%	66.67%	55.56%	75.00%	87.50%
T	0.00%	83.33%	0.00%	50.00%	76.92%
F	0.00%	41.67%	36.36%	66.67%	60.87%
V	91.67%	100.00%	81.82%	96.00%	88.00%
LED	14.29%	56.00%	72.73%	80.00%	69.23%
S	71.43%	80.00%	60.87%	70.59%	75.00%
QA	60.00%	82.35%	85.71%	100.00%	92.31%
JP	0.00%	0.00%	0.00%	25.00%	44.44%
Macro Avg	62.83%	72.70%	65.64%	79.41%	81.58%
Weighted Avg	91.50%	92.98%	91.53%	93.71%	95.22%

color information from the segmentation mask for the corresponding label value in the bounding box. The number of bounding boxes with the same label value gives the number of specific Components in the PCB. For getting the ground truth component count, we use the ground truth mask image instead of the predicted mask and the same steps mentioned above.

Fig. 8 compares the ground truth component count with the predicted component count. As given in Fig. 8(b), we found 3 Jumper Links (JP), 5 Vacuum Tube (V), 76 Capacitors (C), 40 Resistors (R), 8 Buttons (BTN), 2 Connectors (J), 4 Switches (SW), 3 Integrated Circuits (IC), and 1 Ferrite Bead (FB). Our model predicts a total of 142 components in Fig. 8(b), which is same as the component count obtained from the ground truth image 8(a). Similarly for Fig. 8(c) and (d), the individual component counts predicted by our model is same as that seen from the ground truth.

7. Ablation study and failure case analysis

In this section, we provide deeper insights by presenting ablation results (Sections 7.1–7.2) and analyzing the images that are misclassified by our network (Section 7.3).

7.1. Quantitative ablation results

We now verify the contribution of each component in our model by individually removing each component and showing the predictive performance. Table 7 shows the results. For every serial number in Table 7, its explanation in the text has the same serial number.

1. PCBSegClassNet: This shows the full network with all the optimizations and hence, it has the highest predictive performance.

2. PCBSegClassNet (With RGB color space): Our whole network uses HSI + CLAHE processed images. Using RGB images (and no CLAHE), we achieve a DICE score of 93.03% and an IoU score of 80.27%. This confirms the effectiveness of the HSI color space and CLAHE processing.

3. PCBSegClassNet (Without CLAHE): On excluding CLAHE pre-processing, the DICE score reduces to 94.96%, and the IoU score reduces to 91.45%. This confirms the usefulness of the CLAHE processing.

Table 7

Ablation results.

S.No	Model	DICE	IoU
1	PCBSegClassNet	96.29%	92.74%
Preprocessing related ablation			
2	On RGB space (instead of HSI+CLAHE)	93.03%	80.27%
3	Without CLAHE	94.96%	91.45%
Pre-training related ablation			
4	Without N-pair contrastive loss	88.05%	79.71%
5	Without super-resolution (EDSR)	90.41%	77.88%
Network architecture related ablation			
6	Without TEM	93.34%	60.69%
Loss-related ablation			
7	With only DICE loss	95.49%	87.79%
8	With only IoU loss	95.11%	90.31%
9	With only SSIM loss	93.28%	82.40%

4. PCBSegClassNet (Without N-pair contrastive loss): To better learn segmentation embeddings, we have pre-trained PCBSegClassNet using N-pair contrastive loss on the component images (Section 4.5). Here, we present segmentation results obtained without using contrastive loss during pre-training. Our results of 88.05% and 79.71% on the DICE score and IoU (respectively) demonstrate the usefulness of contrastive learning.

5. PCBSegClassNet (Without super-resolution (EDSR)): As discussed in Section 4.5, we pretrain our segmentation network on high-resolution images obtained from the EDSR super-resolution network. On not using the super-resolution network, the DICE score reduces by 6.1 percentage points to just 90.41%. This finding emphasizes the significance of high-resolution images in learning embedding representation.

6. PCBSegClassNet (Without TEM): On removing the texture enhancement module (TEM) from the decoder, the IoU score degrades from 93.40% to 60.69%. Without TEM, our network cannot learn semantic texture information from the borders and edges. Hence, it is unable to shape the component appropriately. This underscores the importance of our proposed TEM module.

7. PCBSegClassNet (With only DICE loss): Our full network is trained on the proposed combined DIS loss. We now show the results of the network that is trained only using the DICE loss. As shown in Table 7, this network achieves a high DICE score (95.49%), but a much lower IoU score (87.79%) than our full network (trained on the combined loss). Evidently, our proposed combined loss significantly improves predictive performance, particularly in terms of boundary quality.

8. PCBSegClassNet (With only IoU loss): Now, we present the results of the network that is trained only using the IoU loss. As shown in Table 7, this network achieves a DICE score of 95.11% and an IoU score of 90.31%. This network has shown a higher IoU value than the one trained with only DICE loss and only SSIM loss. Still, it is inferior to the network trained with combined loss.

9. PCBSegClassNet (With only SSIM loss): As shown in Table 7, the network trained with only SSIM loss achieves a DICE score of 93.28% and an IoU score of 82.40%. These results are almost ten percentage points lower than the network trained on the combined loss.

7.2. Qualitative ablation results

Fig. 9 visually compares the results of network trained with different loss functions. Fig. 9(a) shows the original image and Fig. 9(b) shows the ground truth segmentation mask. Fig. 9(c) displays the predicted mask with the combined DIS loss, Fig. 9(d), (e), (f) show the generated mask with DICE, IoU, and SSIM loss values, respectively. Evidently, our proposed combined DIS loss is more effective than the individual loss functions. Fig. 9(d) shows the missing objects; the network trained with only DICE loss misses almost all the pins from the IC. Fig. 9(e) shows

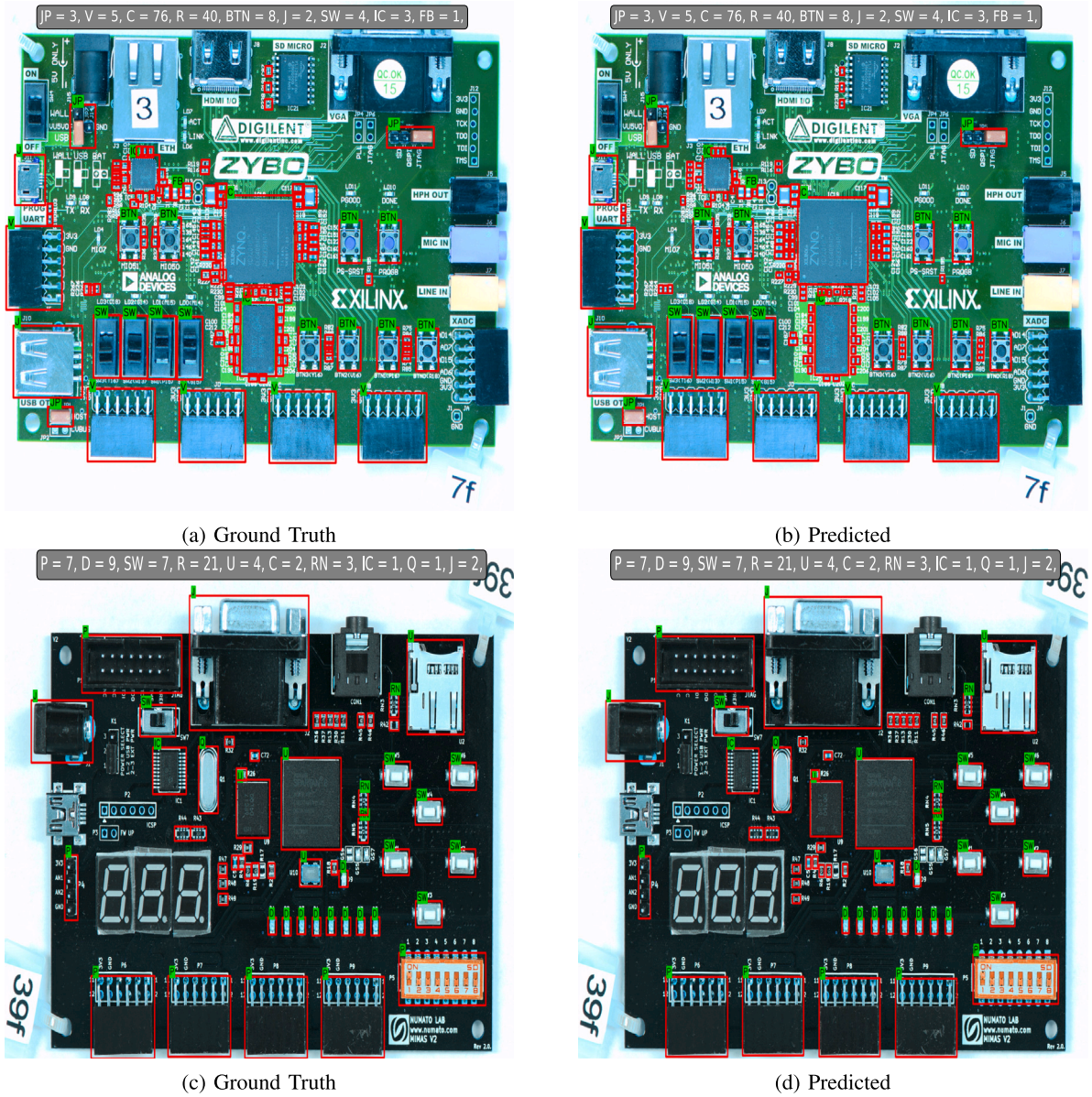


Fig. 8. PCBSegClassNet result of object detection and component counting on two sample images. The number of components found is shown at the top of the figure.

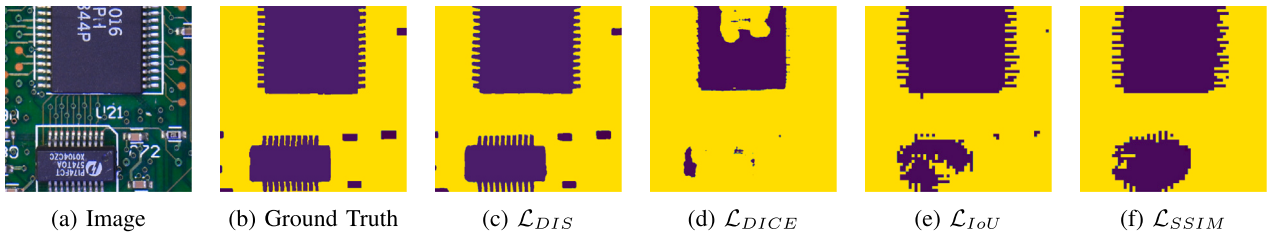


Fig. 9. (a) Original image (b) ground-truth mask (c)–(f) masks produced by PCBSegClassNet when trained with different loss functions.

the prediction with closer output to ground truth compared to DICE loss prediction. The network trained with IoU loss has more control over the border; however, the structure of the second IC is not retained. Fig. 9(f) shows the prediction mask with more structure and shape information retained compared to DICE and IoU loss.

7.3. Analysis of failure cases in classification

To gain deeper insights, we now analyze some images that our model incorrectly classifies. Fig. 10 shows four such examples of PCB

components. Fig. 10(a) has the ground truth of “IC”, but our model predicts “U”. Both “IC” and “U” are different types of integrated circuits, and this can be a reason for misclassification. In Fig. 10(b) and (c), the ground-truth is “R” and “C” respectively, however, our model predicts them as “RN” and “FB”. One possible reason for this misclassification could be that objects such as “inductor”, “ferrite bead” and “capacitor” have similar visual characteristics, as shown by the dataset creators (Jessurun et al., 2022). Similarly, in Fig. 10(d), the ground truth is “U”, but our model mispredicts it as “M”. This may

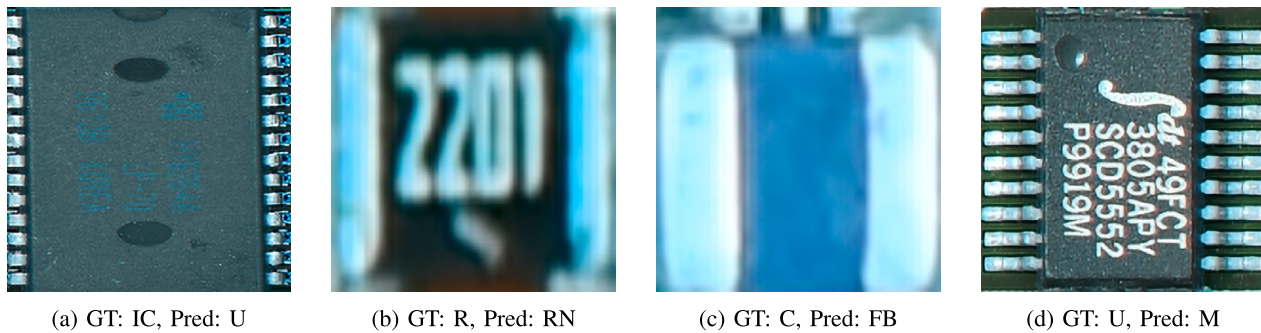


Fig. 10. Failure case scenarios (GT = ground truth, Pred = prediction).

be because some of the “M” class images have similar characteristics as “U” class images. Overall, our model fails for very complicated and similar-looking images.

8. Conclusion

E-waste is leading to a fast-rising global disposal issue. Although PCBs use high-value elements, their widely varying composition poses challenges in waste PCB recycling. To solve these challenges, we propose a novel deep-neural network named PCBSegClassNet in this paper. PCBSegClassNet can perform both PCB component classification and segmentation. For training our model, we propose a combined DIS loss function. Using the combined loss, PCBSegClassNet is able to build segmentation probability maps with precise borders that capture coarse and fine structures, such as thin areas and minor components.

We evaluate PCBSegClassNet on publicly available FPIC dataset (Jessurun et al., 2022) (refer Section 3.1 for more details). We have extensively compared our technique against state-of-the-art (SOTA) open-source models. We observe that for segmentation, PCBSegClassNet achieves higher IoU than all previous models, such as UNet (Ronneberger et al., 2015), DeepLabv3+ (Chen et al., 2017), FPN (Lin et al., 2017), PSPNet (Zhao et al., 2017), SegNet (Badrinarayanan et al., 2017), PVT-small (Wang et al., 2021), PVTv2-B0 (Wang et al., 2022), and Swin-Transformer (Liu et al., 2021). Similarly, for classification, PCBSegClassNet outperforms all SOTA models (He et al., 2016; Huang et al., 2017; Sandler et al., 2018; Tan & Le, 2019) on accuracy, precision, recall, MCC (Chicco & Jurman, 2020), and F1-score. Specifically, PCBSegClassNet achieves 96.3% DICE score and 92.7% IoU for the segmentation task and 95.2% accuracy for the classification task. The ablation results have shed light on the roles played by various components of the network and the loss function.

PCBSegClassNet segmentation network has a model size of 5.58 MB and the classification network has a model size of 3.7 MB. Our network achieves a throughput of 122 frames-per-second on RTX 2080Ti GPU for segmentation of 512×512 size images. Thus, PCBSegClassNet is light-weight and works in real-time.

Our future work will focus on using unsupervised learning for feature extraction. We also plan to combine multiple datasets to create a more extensive component database. This will allow us to improve the efficacy of our network for components with low sample counts. Further, we would like to use an unsupervised learning approach to cover all 58 component classes. We will integrate our proposed TEM in the existing networks and study the improvement in predictive performance. By changing the decoder network, our proposed model can also be extended to other tasks.

CRedit authorship contribution statement

Dhruv Makwana: Conceptualization, Methodology, Software, Validation, Writing – original draft, Writing – review & editing, Visualization. **Sai Chandra Teja R.:** Conceptualization, Analysis & investigation, Writing – original draft, Writing – review & editing, Project

administration-. **Sparsh Mittal:** Methodology, Resources, Writing – review & editing, Supervision, Project administration, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Code is available at <https://github.com/CandleLabAI/PCBSegClassNet>.

References

- Agustsson, E., & Timofte, R. (2017). Ntire 2017 challenge on single image super-resolution: Dataset and study. In *Proceedings of the IEEE conference on computer vision and pattern recognition workshops* (pp. 126–135).
- Awasthi, A. K., Zlamparet, G. I., Zeng, X., & Li, J. (2017). Evaluating waste printed circuit boards recycling: Opportunities and challenges, a mini review. *Waste Management & Research*, 35(4), 346–356.
- Badrinarayanan, V., Kendall, A., & Cipolla, R. (2017). SegNet: A deep convolutional encoder-decoder architecture for image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(12), 2481–2495.
- Barnwal, A., & Dhawan, N. (2020). Physical processing of discarded integrated circuits for recovery of metallic values. *JOM*, 72(7), 2730–2738.
- Chen, L.-C., Papandreou, G., Kokkinos, I., Murphy, K., & Yuille, A. L. (2017). DeepLab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected CRFs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(4), 834–848.
- Cheong, L. K., Suandi, S. A., & Rahman, S. (2019). Defects and components recognition in printed circuit boards using convolutional neural network. In *10th international conference on robotics, vision, signal processing and power applications* (pp. 75–81). Springer.
- Chicco, D., & Jurman, G. (2020). The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21(1), 1–13.
- Chollet, F. (2017). Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 1251–1258).
- Fidon, L., Li, W., Garcia-Peraza-Herrera, L. C., Ekanayake, J., Kitchen, N., Ourselin, S., et al. (2017). Generalised Wasserstein dice score for imbalanced multi-class segmentation using holistic convolutional networks. In *International MICCAI Brainlesion workshop* (pp. 64–76). Springer.
- Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feed-forward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics* (pp. 249–256). JMLR Workshop and Conference Proceedings.
- Hadi, P., Xu, M., Lin, C. S., Hui, C.-W., & McKay, G. (2015). Waste printed circuit board recycling techniques and product utilization. *Journal of Hazardous Materials*, 283, 234–243.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770–778).
- Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., et al. (2017). Mobilenets: Efficient convolutional neural networks for mobile vision applications. arXiv preprint [arXiv:1704.04861](https://arxiv.org/abs/1704.04861).

- Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 4700–4708).
- Jessurun, N., Dizon-Paradis, O. P., Harrison, J., Ghosh, S., Tehranipoor, M. M., Woodard, D. L., et al. (2022). FPIC: A novel semantic dataset for optical PCB assurance. *arXiv preprint arXiv:2202.08414*.
- Júnior, A. A., Silva, L. H. d. S., Fernandes, B. J., Azevedo, G. O., & Oliveira, S. C. (2022). Learning from pseudo-labels: Self-training electronic components detector for waste printed circuit boards. In *2022 35th SIBGRAPI conference on graphics, patterns and images, Vol. 1 SIBGRAPI*, (pp. 252–257). IEEE.
- Kiddee, P., Naidu, R., & Wong, M. H. (2013). Electronic waste management approaches: An overview. *Waste Management*, 33(5), 1237–1250.
- Kim, J.-O., Lee, Y.-A., & Park, T.-H. (2012). Automatic extraction of component inspection regions from printed circuit board. In *2012 IEEE/SICE international symposium on system integration SII*, (pp. 871–876).
- Knoth, R., Hoffmann, M., Kopacek, B., & Kopacek, P. (2001). Intelligent disassembly of electronic equipment. In *Second international symposium on environmentally conscious design and inverse manufacturing* (pp. 557–561).
- Kuo, C.-W., Ashmore, J. D., Huggins, D., & Kira, Z. (2019). Data-efficient graph embedding learning for PCB component detection. In *IEEE winter conference on applications of computer vision (WACV)* (pp. 551–560). IEEE.
- Ledig, C., Theis, L., Huszár, F., Caballero, J., Cunningham, A., Acosta, A., et al. (2017). Photo-realistic single image super-resolution using a generative adversarial network. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 4681–4690).
- Lee, C.-H., Chang, C.-T., Fan, K.-S., & Chang, T.-C. (2004). An overview of recycling and treatment of scrap computers. *Journal of Hazardous Materials*, 114(1–3), 93–100.
- Li, W., Esders, B., & Breier, M. (2013). SMD segmentation for automated PCB recycling. In *11th IEEE international conference on industrial informatics INDIN*, (pp. 65–70). IEEE.
- Li, J., Gu, J., Huang, Z., & Wen, J. (2019). Application research of improved YOLO V3 algorithm in PCB electronic component detection. *Applied Sciences*, 9(18), 3750.
- Li, W., Jiang, C., Breier, M., & Merhof, D. (2016). Localizing components on printed circuit boards using 2d information. In *2016 IEEE international conference on industrial technology ICIT*, (pp. 769–774). IEEE.
- Li, J., Li, W., Chen, Y., & Gu, J. (2021). A PCB electronic components detection network design based on effective receptive field size and anchor size matching. *Computational Intelligence and Neuroscience*, 2021.
- Li, D., Li, C., Chen, C., & Zhao, Z. (2020). Semantic segmentation of a printed circuit board for component recognition based on depth images. *Sensors*, 20(18).
- Li, J., Shrivastava, P., Gao, Z., & Zhang, H.-C. (2004). Printed circuit board recycling: a state-of-the-art survey. *IEEE Transactions on Electronics Packaging Manufacturing*, 27(1), 33–42.
- Li, D., Xu, L., Ran, G., & Guo, Z. (2021). Computer vision based research on PCB recognition using SSD neural network. In *Journal of physics: Conference series, Vol. 1815*. IOP Publishing, Article 012005.
- Lim, D.-u., Kim, Y.-G., & Park, T.-H. (2019). SMD classification for automated optical inspection machine using convolution neural network. In *Third IEEE international conference on robotic computing IRC*, (pp. 395–398).
- Lim, B., Son, S., Kim, H., Nah, S., & Mu Lee, K. (2017). Enhanced deep residual networks for single image super-resolution. In *Proceedings of the IEEE conference on computer vision and pattern recognition workshops* (pp. 136–144).
- Lin, T.-Y., Dollár, P., Girshick, R., He, K., Hariharan, B., & Belongie, S. (2017). Feature pyramid networks for object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 2117–2125).
- Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., et al. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 10012–10022).
- Lu, H., Mehta, D., Paradis, O., Asadizanjani, N., Tehranipoor, M., & Woodard, D. L. (2020). FICS-PCB: A multi-modal image dataset for automated printed circuit board visual inspection. *Cryptology ePrint Archive*.
- Luo, W., & Yang, M. (2020). Semi-supervised semantic segmentation via strong-weak dual-branch network. In *European conference on computer vision* (pp. 784–800). Springer.
- Mallaiyan Sathiaselalan, M. A., Paradis, O. P., Taheri, S., & Asadizanjani, N. (2021). Why is deep learning challenging for printed circuit board (PCB) component recognition and how can we address it? *Cryptography*, 5(1), 9.
- Mantravadi, A., Makwana, D., Teja, R. S. C., Mittal, S., & Singhal, R. (2023). Dilated involutonal pyramid network (DInPNet): A novel model for printed circuit board (PCB) components classification. In *24th international symposium on quality electronic design (ISQED)*. California, USA.
- Máttyus, G., Luo, W., & Urtasun, R. (2017). Deeproadmapper: Extracting road topology from aerial images. In *Proceedings of the IEEE international conference on computer vision* (pp. 3438–3446).
- Mazzini, D. (2018). Guided upsampling network for real-time semantic segmentation. In *British machine vision conference (BMVC)*.
- Nag, S., Makwana, D., R, S. C. T., Mittal, S., & Mohan, C. (2022). WaferSegClassNet - A light-weight network for classification and segmentation of semiconductor wafer defects. *Computers in Industry*, 142, Article 103720.
- Ouali, Y., Hudelot, C., & Tami, M. (2020). Semi-supervised semantic segmentation with cross-consistency training. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 12674–12684).
- Pasunuri, A., Jessurun, N., Dizon-Paradis, O. P., & Asadizanjani, N. (2021). A comparison of neural networks for PCB component segmentation. In *IEEE international symposium on hardware oriented security and trust (HOST)* (pp. 113–123). <http://dx.doi.org/10.1109/HOST49136.2021.9702286>.
- Polat, S., Tremau, A., & Boochs, F. (2021). Combined use of 3D and HSI for the classification of printed circuit board components. *Applied Sciences*, 11(18), 8424.
- Pramerdorfer, C., & Kampel, M. (2015a). A dataset for computer-vision-based PCB analysis. In *2015 14th IAPR international conference on machine vision applications MVA*, (pp. 378–381).
- Pramerdorfer, C., & Kampel, M. (2015b). PCB recognition using local features for recycling purposes. In *VISAPP* (3) (pp. 71–78).
- Putera, S. I., & Ibrahim, Z. (2010). Printed circuit board defect detection using mathematical morphology and MATLAB image processing tools. In *2nd international conference on education technology and computer, Vol. 5* (pp. V5–359–V5–363). IEEE.
- Reza, M. A., Chen, Z., & Crandall, D. J. (2020). Deep neural network-based detection and verification of microelectronic images. *Journal of Hardware and Systems Security*, 4(1), 44–54.
- Ronneberger, O., Fischer, P., & Brox, T. (2015). U-net: Convolutional networks for biomedical image segmentation. In *International conference on medical image computing and computer-assisted intervention* (pp. 234–241). Springer.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 4510–4520).
- Silva, L. H. d. S., Júnior, A. A., Azevedo, G. O., Oliveira, S. C., & Fernandes, B. J. (2021). Estimating recycling return of integrated circuits using computer vision on printed circuit boards. *Applied Sciences*, 11(6), 2808.
- Sohn, K. (2016). Improved deep metric learning with multi-class N-pair loss objective. *Advances in Neural Information Processing Systems*, 29.
- Tan, M., & Le, Q. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning* (pp. 6105–6114). PMLR.
- Wang, Z., Simoncelli, E. P., & Bovik, A. C. (2003). Multiscale structural similarity for image quality assessment. In *The thirty-seventh asilomar conference on signals, systems & computers, 2003, Vol. 2* (pp. 1398–1402). Ieee.
- Wang, W., Xie, E., Li, X., Fan, D.-P., Song, K., Liang, D., et al. (2021). Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 568–578).
- Wang, W., Xie, E., Li, X., Fan, D.-P., Song, K., Liang, D., et al. (2022). PVTv2: Improved baselines with pyramid vision transformer. *Computational Visual Media*, 8(3), 1–10.
- Weinert, K., Pandey, V., Naranjo Corona, S., & Danielewski, A. (2017). Autonomous waste characterization in environmentally conscious decision making. In *International design engineering technical conferences and computers and information in engineering conference, Vol. 58165*. American Society of Mechanical Engineers, V004T05A035.
- Zeng, Z., Xie, W., Zhang, Y., & Lu, Y. (2019). RIC-Unet: An improved neural network based on Unet for nuclei segmentation in histology images. *IEEE Access*, 7, 21420–21428.
- Zhang, Z., & Sabuncu, M. (2018). Generalized cross entropy loss for training deep neural networks with noisy labels. *Advances in Neural Information Processing Systems*, 31.
- Zhao, H., Shi, J., Qi, X., Wang, X., & Jia, J. (2017). Pyramid scene parsing network. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 2881–2890).
- Zhu, L., Ji, D., Zhu, S., Gan, W., Wu, W., & Yan, J. (2021). Learning statistical texture for semantic segmentation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 12537–12546).
- Zuiderveld, K. (1994). Contrast limited adaptive histogram equalization. In *Graphics gems* (pp. 474–485). Academic Press, Retrieved from <https://cir.nii.ac.jp/crid/1571698601099987968>.